

1. PROJECT OVERVIEW

- **Project Title:** ComicCraft - AI Comic Story Creator using Gemini Models

2. ARCHITECTURAL OVERVIEW :

ComicCraft is engineered using a highly modular, decoupled three-tier architecture. This separates the Client Presentation (Frontend), Application Logic (Backend), and Cloud AI Services, ensuring that the software is scalable, maintainable, and efficient in handling asynchronous requests.

3. CLIENT PRESENTATION TIER (FRONTEND)

- **Core Technologies:** HTML5, CSS3, Jinja2 Templating.
- **Role:** Manages user interaction and visual output.
- **Components:**
 - `index.html`: The primary data collection form capturing storytelling parameters.
 - `preview.html`: The dynamic template that iterates through backend dictionary data to render the comic panels and narrative text sequentially.

4. APPLICATION LOGIC TIER (BACKEND)

- **Core Technologies:** Python 3.10, FastAPI, Uvicorn Server.
- **Role:** Acts as the central orchestrator. It receives user inputs, communicates with AI endpoints, and compiles the final layout.
- **Modular Scripts:**
 - `main_routes.py`: Contains the POST/GET endpoints and controls the execution sequence of the AI pipeline.
 - `gemini_handler.py`: Manages HTTPS requests to the Google Gemini API for outlining and dialogue creation.

- `image_handler.py`: Executes asynchronous calls to the Hugging Face Inference API for image rendering, equipped with robust failover mechanisms for network errors.
- `layout_manager.py`: A utility module that maps the generated text strings to their corresponding downloaded image paths.
- `pdf_compiler.py`: Utilizes the FPDF library to calculate page coordinates and draw the text and images onto a downloadable PDF document.

5. CLOUD AI INTEGRATION TIER The application depends on external cloud processing for generative tasks, communicating securely via standard HTTPS protocols (`urllib.request`).

- **Text Processing:** Google Generative AI (Gemini Flash & Pro).
- **Image Processing:** Hugging Face Stable Diffusion v1.5.
- **Security:** All API credentials are injected dynamically at runtime using `python-dotenv`, preventing key exposure in the source code.

6. DATA & FILE HANDLING The system is designed to be lightweight and stateless, avoiding heavy database dependencies for temporary generation tasks.

- **Image Storage:** Visual panels (`.jpg`) are saved temporarily in the `static/panels/` directory.
- **Document Storage:** Compiled PDF files are temporarily hosted in the `static/exports/` directory for user download.

7. EXECUTION SEQUENCE

1. User submits HTML Form → `main_routes.py`.
2. Backend queries `gemini_handler.py` → Receives Story Outline.
3. Backend queries `gemini_handler.py` → Receives Full Dialogue.
4. Backend iteratively queries `image_handler.py` → Saves Images.
5. `layout_manager.py` combines text and image paths.
6. `pdf_compiler.py` generates the PDF.

7. Frontend renders `preview.html` with the final data.